Understanding Agentic AI Systems:

LangChain, LangGraph, Agent Lifecycle, and MCP

Hobhav S Fulzele

November 6, 2026

Contents

1 Introduction to LangChain **2**

1.1 Main Components of LangChain 2

1.2 Indexes 3

2 Introduction to LangGraph **4**

2.1 Core Concepts of LangGraph 5

2.2 Important Features of LangGraph 6

3 AI Agent Development Lifecycle (ADLC) **6**

3.1 Introduction to Agent Development Lifecycle 6

3.2 Phases of the Agent Development Lifecycle 7

3.3 Importance of the Agent Development Lifecycle 12

4 Model Context Protocol (MCP) **13**

4.1 Introduction to Model Context Protocol 13

4.2 MCP Architecture 13

4.3 Core Components Provided by MCP Servers 14

4.4 MCP Communication and Transport 15

4.5 Working Process of MCP 16

4.6 Security Considerations 16

1 Introduction to LangChain

LangChain is an open source orchestration framework for application development using large language models (LLMs). Available in both Python- and Javascript-based libraries, LangChain's tools and APIs simplify the process of building LLM-driven applications like chatbots and AI agents.

An LLM can understand and generate human-like text. However, real-world applications often require additional capabilities such as document processing, database access, external API integration, memory management, and tool execution.

LangChain provides reusable components that help developers connect these capabilities.

1.1 Main Components of LangChain

1.1.1 Language Models

Language models are the core components of an LLM-powered application. They process input and generate responses.

LangChain supports integrations with different model providers, including OpenAI, Anthropic, Google, and other model platforms.

Chat models are designed for conversational interactions, whereas embedding models convert text into numerical representations called embeddings.

1.1.2 Prompt Templates

Prompts are the instructions given to an LLM. The "art" of composing prompts that effectively provide the context necessary for the LLM to interpret input and structure output in the way most useful to you is often called prompt engineering.

The PromptTemplate class in LangChain formalizes the composition of prompts without the need to manually hard code context and queries. Important elements of a prompt are likewise entered as formal classes, like input_variables. A prompt template can thus contain and reproduce context, instructions (like "do not use technical terms"), a set of examples to guide its responses (in what is called "few-shot prompting"), a specified output format or a standardized question to be answered.You can save and name an effectively structured prompt template and easily reuse it as needed.

Though these elements can all be manually coded, PromptTemplate modules empower smooth integration with other LangChain features, like the eponymous chains.

1.1.3 Chains

As its name implies, chains are the core of LangChain's workflows. They combine LLMs with other components, creating applications by executing a sequence of functions.

The most basic chain is LLMChain. It simply calls a model and prompt template for that model. For example, imagine you saved a prompt as "ExamplePrompt" and wanted to run it against Flan-T5. You can import LLMChain from langchain.chains, then define chain_example = LLMChain(llm = flan-t5, prompt = ExamplePrompt). To run the chain for a given input, you simply call chain_example.run("input").

To use the output of one function as the input for the next function, you can use SimpleSequentialChain. Each function could utilize different prompts, different tools, different parameters or even different models, depending on your specific needs.

1.1.4 Tools

Tools allow an AI application to interact with external systems.

- **Wolfram Alpha**: provides access to powerful computational and data visualization functions, enabling sophisticated mathematical capabilities.
- **Google Search**: provides access to Google Search, equipping applications and agents with real-time information.
- **OpenWeatherMap**: fetches weather information.
- **Wikipedia**: provides efficient access to information from Wikipedia articles.

An AI agent can select an appropriate tool depending on the user's request.

1.1.5 Memory

LLMs, by default, do not have any long-term memory of previous interactions (unless that chat history is used as input for a query). LangChain solves this problem with simple utilities for adding memory to a system, with options ranging from retaining the entirety of all conversations to retaining a summarization of the conversation thus far to retaining the n most recent exchanges.

1.2 Indexes

To achieve certain tasks, LLMs will need access to specific external data sources not included in its training dataset, such as internal documents, emails or datasets. LangChain

collectively refers to such external documentation as “indexes”.

1.2.1 Document loaders

LangChain offers a wide variety of document loaders for third party applications. This allows for easy importation of data from sources like file storage services (like Dropbox, Google Drive and Microsoft OneDrive), web content (like YouTube, PubMed or specific URLs), collaboration tools (like Airtable, Trello, Figma and Notion), databases (like Pandas, MongoDB and Microsoft), among many others.

1.2.2 Vector databases

Vector databases represent data points by converting them into vector embeddings: numerical representations in the form of vectors with a fixed number of dimensions, often clustering related data points using unsupervised learning methods. This enables low latency queries, even for massive datasets, which greatly increases efficiency. Vector embeddings also store each vector’s metadata, further enhancing search possibilities.
LangChain provides integrations for over 25 different embedding methods, as well as for over 50 different vector stores (both cloud-hosted and local).

1.2.3 Text splitters

LangChain’s TextSplitters split text up into small, semantically meaningful chunks that can then be combined using methods and parameters of your choosing.

1.2.4 Retrieval

With LangChain, we can also build agentic RAG systems. In traditional RAG applications, the LLM is provided with a vector database to reference when forming its responses. In contrast, agentic AI applications are not restricted to only data retrieval. Agenic RAG can also encompass tools for tasks such as solving mathematical calculations, writing emails, performing data analysis and more.

2 Introduction to LangGraph

LangGraph is a framework for building stateful, multi-step AI agent workflows using graph-based execution.

While LangChain provides reusable components for LLM applications, LangGraph focuses on workflow orchestration and execution control.

Traditional sequential chains generally execute operations one after another.

However, complex AI agents may require conditional execution, repeated operations, state management, human approval, and interaction between multiple agents.

LangGraph provides mechanisms to implement these requirements.

2.1 Core Concepts of LangGraph

2.1.1 Nodes

Nodes represent individual processing steps in a workflow.

A node may execute an LLM call, invoke an external tool, retrieve information, or perform a validation operation.

2.1.2 Edges

Edges define the connections between nodes.

They determine which operation should execute next.

Two important types of edges are:

- Normal edges: Define a direct transition between two nodes.
- Conditional edges: Select the next node according to a condition or decision.

2.1.3 State

State represents the information maintained during workflow execution.

It may contain:

- User input.
- Conversation history.
- Retrieved documents.
- Intermediate results.
- Tool execution outputs.

- Workflow status.

2.2 Important Features of LangGraph

- **Stateful Execution:** Maintains information throughout workflow execution.
- **Conditional Routing:** Selects the next operation based on conditions.
- **Cyclic Workflows:** Allows agents to repeat operations when necessary.
- **Checkpointing:** Saves workflow state to support recovery and resumption.
- **Human-in-the-Loop:** Supports human review and approval.
- **Multi-Agent Support:** Enables coordination between specialized agents.

3 AI Agent Development Lifecycle (ADLC)

3.1 Introduction to Agent Development Lifecycle

The Agent Development Lifecycle (ADLC) is a structured, end-to-end methodology for developing, testing, deploying, operating, and managing AI agents. It provides a systematic approach to building reliable, secure, scalable, and maintainable agentic AI systems.

Traditional software development methodologies are primarily designed for deterministic applications, where software follows predefined instructions. However, AI agents powered by Large Language Models (LLMs) exhibit probabilistic behavior. They can reason, make decisions, interact with external tools, and dynamically determine how to accomplish a particular objective.

Therefore, developing AI agents requires additional considerations such as behavioral evaluation, security guardrails, observability, continuous monitoring, and performance optimization.

The ADLC addresses these requirements by dividing the development process into six interconnected phases:

1. Plan
2. Code and Build
3. Test and Release
4. Deploy

5. Operate
6. Monitor

These phases provide a continuous development and management approach that helps organizations maintain the reliability, security, and effectiveness of AI agents.

3.2 Phases of the Agent Development Lifecycle

3.2.1 Phase 1: Plan

Planning is the initial phase of the Agent Development Lifecycle. It focuses on identifying the business problem, defining the objectives, understanding the requirements, and determining whether an AI agent is the appropriate solution.

Before developing an agent, organizations should clearly understand what they want the system to achieve and how its success will be measured.

The important activities involved in this phase include:

- **Identify Use Cases:** Determine the specific business problems that require automation, reasoning, or decision-making.
- **Define Objectives:** Establish clear goals and expected outcomes for the AI agent.
- **Define Success Metrics:** Identify Key Performance Indicators (KPIs) such as accuracy, task completion rate, response time, and user satisfaction.
- **Identify Required Resources:** Determine the tools, databases, APIs, knowledge bases, and external services required by the agent.
- **Define Agent Behavior:** Document the expected behavior, responsibilities, constraints, and operating procedures.
- **Assess Feasibility:** Evaluate whether an AI agent is necessary or whether traditional automation or a simpler LLM-based application can solve the problem effectively.

For example, an organization planning to develop an AI customer support agent must first identify the types of customer queries, required data sources, expected response quality, and business objectives.

Proper planning ensures that the agent is developed according to clearly defined requirements.

3.2.2 Phase 2: Code and Build

After completing the planning phase, developers begin designing and implementing the AI agent.

This phase involves selecting appropriate language models, designing prompts, implementing agent logic, integrating tools, and establishing memory and orchestration mechanisms.

The major activities include:

- **Model Selection:** Select suitable Large Language Models (LLMs) based on reasoning capabilities, accuracy, computational cost, and response latency.
- **Prompt Engineering:** Design system instructions and prompts that define the agent's behavior, role, objectives, and constraints.
- **Agent Orchestration:** Implement the workflow that determines how the agent processes requests, makes decisions, and executes tasks.
- **Memory Management:** Implement suitable memory mechanisms to maintain relevant information and conversation context.
- **Tool Integration:** Connect external tools, databases, APIs, and enterprise services required for task execution.
- **Version Control:** Maintain versions of prompts, models, agent configurations, and orchestration logic to support reproducibility.
- **Security Implementation:** Introduce authentication, authorization, sandboxing, and access restrictions during development.

Model Context Protocol (MCP) can be used to standardize connections between AI applications and external tools and resources.

Frameworks such as LangChain and LangGraph can also be used to implement the components and workflow orchestration required by an agent.

For example, an AI research assistant can be developed using an LLM for reasoning, a web search tool for information retrieval, and a vector database for accessing relevant documents.

This phase transforms the requirements defined during planning into a functional AI agent.

3.2.3 Phase 3: Test and Release

Once the agent has been developed, it must undergo structured testing and evaluation before being released into a production environment.

Unlike traditional software testing, AI agent testing must evaluate not only technical correctness but also the agent's behavior, reasoning quality, tool usage, and ability to achieve the intended objectives.

The important activities include:

- **Functional Testing:** Verify that the agent performs its assigned tasks correctly and that all integrated components function as expected.
- **Behavioral Testing:** Evaluate the agent's responses and actions across different user inputs and scenarios.
- **Performance Evaluation:** Measure task completion rate, response accuracy, latency, and computational cost.
- **Security Testing:** Identify vulnerabilities involving unauthorized access, sensitive information, and unsafe tool execution.
- **Policy Validation:** Ensure that the agent follows predefined organizational policies and behavioral restrictions.
- **Red Teaming:** Simulate adversarial inputs and potentially harmful situations to identify weaknesses in the agent's security and behavior.
- **Human Evaluation:** Use human reviewers to assess the quality, reliability, and appropriateness of the agent's outputs.

Evaluation can involve predefined test datasets, automated assessment techniques, and human-in-the-loop review processes.

For example, a customer support agent should be tested using different customer queries, including ambiguous questions, incorrect information, and requests involving sensitive account details.

The agent should be released only after it satisfies the required performance, safety, and quality criteria.

The Code and Build phase and Test and Release phase form an iterative development loop. If testing identifies problems, developers can modify the agent and repeat the testing process.

3.2.4 Phase 4: Deploy

After successful testing and validation, the AI agent is deployed into its intended production environment.

Deployment makes the agent available to users and allows it to interact with real-world systems and workflows.

However, deploying an AI agent requires more than simply making the application available. Organizations must ensure that the agent operates within controlled and secure environments.

The major activities include:

- **Production Deployment:** Integrate the validated agent into the organization's production infrastructure.
- **Progressive Rollout:** Release the agent gradually to manage operational risks and evaluate its behavior under real-world conditions.
- **Sandboxing:** Execute agents within restricted environments to prevent unauthorized access to files, networks, and system resources.
- **Access Control:** Apply appropriate authentication and authorization mechanisms to restrict access to tools and enterprise data.
- **Version Management:** Maintain deployment versions and provide rollback mechanisms in case of failures.
- **Runtime Governance:** Enforce security policies, operational restrictions, and resource limits while the agent is executing.

Common deployment strategies include:

- Canary deployment.
- Rolling deployment.
- Blue-green deployment.

For example, an organization deploying an AI customer support agent may initially release it to a small group of users before making it available to all customers.

This allows developers to identify potential issues and reduce the risk of large-scale failures.

3.2.5 Phase 5: Operate

The ADLC does not end after deployment. Once an AI agent is available in production, it must be continuously operated, observed, and optimized.

The Operate phase focuses on maintaining the agent's effectiveness, reliability, security, and performance under real-world conditions.

The major activities include:

- **Performance Monitoring:** Track operational metrics such as task completion rate, accuracy, latency, and computational cost.
- **Observability:** Collect information about agent execution, tool calls, workflow transitions, errors, and system behavior.
- **Performance Optimization:** Improve prompts, models, tools, and orchestration logic based on observed performance.
- **Feedback Analysis:** Analyze user feedback and operational results to identify areas requiring improvement.
- **Error Management:** Identify failures, investigate their causes, and implement appropriate corrective actions.
- **Continuous Improvement:** Update agent configurations and deploy validated improvements when necessary.

For example, if a deployed customer support agent begins providing inaccurate answers, developers can analyze its execution logs, identify the underlying problem, improve its prompts or knowledge sources, and deploy the updated version.

The Deploy and Operate phases form a continuous feedback loop because operational observations may lead to further improvements and deployments.

3.2.6 Phase 6: Monitor

Monitoring is the final phase of the ADLC. It involves continuously evaluating the agent to ensure that it remains reliable, secure, transparent, fair, and compliant with applicable organizational and regulatory requirements.

Even after successful deployment and optimization, an AI agent may experience changes in performance due to evolving business requirements, changes in data, or model updates.

The major activities include:

- **Continuous Performance Monitoring:** Track the agent's performance and verify that it continues to satisfy predefined objectives.
- **Model Drift Detection:** Identify changes in model behavior or output quality over time.
- **Security Auditing:** Regularly examine the agent's permissions, tool access, and potential vulnerabilities.
- **Fairness and Transparency:** Evaluate whether the agent behaves consistently and provides appropriate explanations for its outputs.
- **Regulatory Compliance:** Ensure that the agent operates according to relevant organizational policies and legal requirements.
- **Documentation and Auditing:** Maintain records of agent ownership, versions, configurations, risk levels, and operational environments.

For example, an AI financial assistant must be continuously monitored to ensure that its recommendations remain accurate, secure, and compliant with applicable requirements.

Monitoring provides the information necessary to identify problems and improve the agent throughout its operational lifetime.

3.3 Importance of the Agent Development Lifecycle

The ADLC provides several important benefits for organizations developing AI agents.

1. **Improved Reliability:** Structured testing and continuous evaluation help identify errors and improve agent performance.
2. **Enhanced Security:** Security controls, sandboxing, and access restrictions reduce operational risks.
3. **Better Performance:** Continuous monitoring helps organizations optimize models, prompts, and tools.
4. **Scalability:** Standardized development and deployment practices support the expansion of agentic systems.
5. **Governance and Compliance:** Documentation, auditing, and policy enforcement improve accountability.
6. **Continuous Improvement:** Feedback from real-world operations enables ongoing optimization of agents.

4 Model Context Protocol (MCP)

4.1 Introduction to Model Context Protocol

The Model Context Protocol (MCP) is an open standard designed to simplify communication between Large Language Models (LLMs), AI applications, and external systems such as databases, APIs, file systems, and software tools. It provides a standardized mechanism through which AI applications can access external information and perform actions using connected services.

An LLM operating independently is primarily limited to generating responses based on the information available in its training data and the context provided during a conversation. It cannot automatically access external databases, interact with applications, or retrieve real-time information unless suitable integrations are provided.

MCP addresses this limitation by defining a common communication protocol between AI applications and external services. Instead of developing a separate integration for every tool, developers can use MCP-compatible clients and servers to establish reusable connections.

MCP can be compared to a USB-C connector: just as USB-C provides a standardized interface for connecting different hardware devices, MCP provides a standardized interface for connecting AI applications to different tools and data sources [**?**].

4.2 MCP Architecture

MCP follows a client-server architecture consisting of three primary components: the MCP Host, MCP Client, and MCP Server. These components work together to establish communication between an AI application and external services.

4.2.1 MCP Host

The MCP Host is the AI application that interacts with the user and manages the overall execution of the AI workflow. It may be an AI assistant, an integrated development environment, or another application that uses an LLM.

The host manages the model interaction and orchestration logic. It also creates and manages MCP clients to communicate with external MCP servers.

Examples of applications that can act as MCP hosts include AI-powered development environments and desktop AI assistants.

4.2.2 MCP Client

The MCP Client is a component inside the host application that establishes and manages communication with an MCP Server. It acts as an intermediary between the host and the server.

An MCP client is responsible for handling protocol communication, managing sessions, processing server responses, and handling communication errors. A host can contain multiple MCP clients to connect to different MCP servers. Each client maintains a connection with a particular server.

4.2.3 MCP Server

The MCP Server exposes external capabilities to an AI application through the MCP standard. It acts as an interface between the AI application and an external system, such as a database, file system, web service, or third-party application.

For example, an MCP server may provide access to GitHub repositories, Slack messages, local files, or database queries. The server makes these capabilities available to MCP clients through standardized protocol operations.

4.3 Core Components Provided by MCP Servers

MCP servers can expose three important types of capabilities: Resources, Tools, and Prompts. Each capability serves a different purpose in providing context and functionality to an AI application.

4.3.1 Resources

Resources provide contextual information to an AI application. They are generally used to expose data that the model or application can retrieve and use during task execution.

Examples include:

- Documents and files.
- Database records.
- Application-specific information.
- Project documentation and configuration data.

Resources primarily support information retrieval rather than performing actions that modify external systems.

4.3.2 Tools

Tools allow an AI application to perform operations through an MCP server. Unlike resources, which primarily expose information, tools can execute operations or interact with external services.

Examples include:

- Executing database queries.
- Sending messages through communication platforms.
- Creating or updating project tasks.
- Retrieving information from external APIs.
- Performing calculations or other application operations.

The LLM or agent determines when a tool is appropriate based on the task and available context. The MCP server handles the corresponding tool operation and returns its result.

4.4 MCP Communication and Transport

MCP uses JSON-RPC 2.0 as its underlying message format for communication between clients and servers. JSON-RPC defines a structured way to exchange requests, responses, and notifications.

The protocol supports the following message types:

- **Requests:** Messages sent by one component to another when an operation or response is required.
- **Responses:** Messages returned to provide the result of a request or report an error.
- **Notifications:** Messages used to communicate events that do not require a response.

MCP communication can use different transport mechanisms. The standard input/output (stdio) transport is commonly used for local integrations, where the client communicates with a locally running server through input and output streams.

For remote communication, MCP supports HTTP-based transport. Earlier MCP implementations commonly used HTTP with Server-Sent Events (SSE), while newer implementations use Streamable HTTP. The transport mechanism determines how messages are transmitted between the client and server.

These communication mechanisms enable MCP to support both local and remote integrations while maintaining a common protocol structure.

4.5 Working Process of MCP

The general working process of MCP can be explained through the following steps:

1. The user submits a request to an AI application acting as the MCP Host.
2. The host processes the request and uses its LLM and orchestration logic to determine whether external information or a tool is required.
3. The MCP Client communicates with the appropriate MCP Server using the MCP protocol.
4. The MCP Server provides the requested resource, executes the requested tool, or supplies an available prompt.
5. The server returns the result to the MCP Client.
6. The client forwards the result to the host, where it becomes part of the context available to the LLM.
7. The LLM uses the returned information to continue its reasoning and generate a response or determine the next action.

This process allows an AI application to interact with external systems through a standardized interface rather than relying on individually implemented tool connections.

4.6 Security Considerations

Although MCP simplifies tool integration, connecting AI applications to external systems introduces security considerations. MCP implementations should apply appropriate security controls to protect data and prevent unauthorized operations.

Important considerations include:

- **Authentication and Authorization:** Verify the identity of clients and restrict access according to user and application permissions.
- **Least Privilege:** Provide each agent or application only the permissions required to perform its intended tasks.
- **Input Validation:** Validate requests and tool arguments before executing operations on external systems.
- **Sensitive Data Protection:** Avoid exposing confidential information unnecessarily and apply appropriate data-handling policies.
- **Human Approval:** Require user confirmation for sensitive or high-impact operations when appropriate.
- **Monitoring and Auditing:** Maintain logs of tool calls, access events, and important operations to support monitoring and troubleshooting.

Consequently, MCP should be implemented with suitable authentication, authorization, validation, and monitoring mechanisms rather than treating protocol compatibility as a substitute for security.